

Some limitations on realizing fault tolerance

Organizing replicated processes into a group helps to increase fault tolerance. However, what should have become clear by now is that there is a price to pay, namely a potential loss of performance. In the solutions discussed so far, processes in a fault-tolerant group may need to exchange numerous messages before reaching a decision. The Byzantine agreement protocol is an excellent illustration of how tightly coupled processes may be. The question that comes to mind is whether realizing specific forms of fault tolerance, like being able to withstand arbitrary failures, is always possible.

On reaching consensus

As we mentioned, if a client can base its decisions through a voting mechanism, we can tolerate that k out of $2k + 1$ processes are lying about their result. The assumption we are making, however, is that processes do not team up to produce a wrong result. In general, matters become more intricate if we demand that a process group reaches consensus, which is needed in many cases. There are three requirements for reaching consensus [Fischer et al., 1985]:

- Processes produce the same output value
- Every output value must be valid
- Every process must eventually provide output

Some examples where reaching consensus is necessary include electing a coordinator, deciding whether or not to commit a transaction, and dividing up tasks among workers. When the communication and processes are all perfect, reaching consensus is often straightforward, but when they are not, problems arise.

The general goal of distributed consensus algorithms is to have all the nonfaulty processes reach consensus on some issue, and to establish that consensus within a finite number of steps. The problem is complicated by the fact that different assumptions about the underlying system require different solutions, assuming solutions even exist. Turek and Shasha [1992] distinguish the following cases:

1. Synchronous versus asynchronous systems. Rephrasing our description somewhat, a system is **synchronous** if and only if the processes are known to operate in a lock-step mode. Formally, this means that there should be some constant $c \geq 1$, such that if any process has taken $c + 1$ steps, every other process has taken at least 1 step.
2. Communication delay is bounded or not. Delay is bounded if and only if we know that every message is delivered with a globally and predetermined maximum time.

3. Message delivery is ordered (in real time) or not. In other words, we distinguish the situation where messages from the different senders are delivered in the order that they were sent in real global time, from the situation in which we do not have such guarantees.
4. Message transmission is done through unicasting or multicasting.

As it turns out, reaching consensus is possible only for the situations shown in Figure 8.17. In all other cases, it can be shown that no solution exists. Note that most distributed systems in practice assume that processes behave asynchronously, message transmission is unicast, and communication delays are unbounded. As a consequence, we need to make use of ordered (reliable) message delivery, such as provided by TCP. And again, in practical situations we assume synchronous behavior to be the default, but take into account that there may be unbounded delays as well. Figure 8.17 illustrates the nontrivial nature of distributed consensus when processes may fail.

		Message ordering				
		Unordered		Ordered		
Process behavior	Synchronous	X	X	X	X	Bounded
				X	X	Unbounded
	Asynchronous				X	Bounded
					X	Unbounded
		Unicast	Multicast	Unicast	Multicast	
Message transmission						

Figure 8.17: Circumstances under which distributed consensus can be reached.

Reaching consensus may not be possible. Fischer et al. [1985] proved that if messages cannot be guaranteed to be delivered within a known, finite time, no consensus is possible if even one process is faulty (albeit if that one process fails silently). The problem with such systems is that arbitrarily slow processes are indistinguishable from crashed ones (i.e., you cannot tell the dead from the living). These and other theoretical results are surveyed by Barborak et al. [1993] and Turek and Shasha [1992].

Consistency, availability, and partitioning

Strongly related to the conditions under which consensus can (not) be reached, is when consistency can be reached. Consistency in this case means that when we have a process group to which a client is sending requests, that the responses returned to that client are correct. We are dealing with a **safety property**: a property that asserts that nothing bad will happen. For

our purposes, the type of operations we consider are those that seem to be executed in a clearly defined order by a single, centralized server. By now, we know better: these operations are executed by a process group in order to withstand the failures of k group members.

We introduced process groups to improve fault tolerance, and, more specifically, to improve availability. Availability is typically a **liveness property**: eventually, something good will happen. In terms of our process groups, we aim to eventually get a (correct) response to every request issued by a client. Being consistent in responses while also being highly available is not an unreasonable requirement for services that are part of a distributed system. Unfortunately, we may be asking too much.

In practical situations, our underlying assumption that the processes in a group can indeed communicate with each other may be false. Messages may be lost; a group may be partitioned due to a faulty network. In 2000, Eric Brewer posed an important theorem which was later proven to be correct by Gilbert and Lynch [2002]:

CAP Theorem: *Any networked system providing shared data can provide only two of the following three properties:*

- **C:** *consistency, by which a shared and replicated data item appears as a single, up-to-date copy*
- **A:** *availability, by which updates will always be eventually executed*
- **P:** *Tolerant to the partitioning of process group (e.g., because of a failing network).*

In other words, in a network subject to communication failures, it is impossible to realize an atomic read/write shared memory that guarantees a response to every request [Gilbert and Lynch, 2012].

This has now become known as the **CAP theorem**, first published as [Fox and Brewer, 1999]. As explained by Brewer [2012], one way of understanding the theorem is to think of two processes unable to communicate because of a failing network. Allowing one process to accept updates leads to inconsistency, so that we can only have properties $\{A, P\}$. If the illusion of consistency is to be provided while the two processes cannot communicate, then one of the two processes will have to pretend to be unavailable, implying having only $\{C, P\}$. However, only if the two processes can communicate, is it possible to maintain both consistency and high availability, meaning that we have only $\{C, A\}$, but no longer property P .

Note also the relationship with reaching consensus; in fact, where consensus requires proving that processes produce the same output, providing consistency is weaker. This also means that if achieving CAP is impossible, then so is consensus.

The CAP theorem is all about reaching a trade-off between safety and liveness, based on the observation that obtaining both in an inherently unreliable system cannot be achieved. Practical distributed systems *are* inherently unreliable. What Brewer and his colleagues observed is that in practical distributed systems, one simply has to make a choice to proceed despite the fact that another process cannot be reached. In other words, we need to do something when a partition manifests itself through high latency.

The bottom line when it seems that partitioning is taking place, is to proceed (tolerating partitions in favor of either consistency or availability), while simultaneously starting a recovery procedure that can mitigate the effects of potential inconsistencies. Exactly deciding on how to proceed is application dependent: in many cases having duplicate keys in a database can easily be fixed (implying that we should tolerate an inconsistency), while duplicate transfers of large sums of money may not (meaning that we should decide to tolerate lower availability). One can argue that the CAP theorem essentially moves designers of distributed systems from theoretical solutions to engineering solutions. The interested reader is referred to [Brewer, 2012] to see how such a move can be made.

Failure detection

It may have become clear from our discussions so far that in order to properly mask failures, we generally need to detect them as well. Failure detection is one of the cornerstones of fault tolerance in distributed systems. What it all boils down to is that for a group of processes, nonfaulty members should be able to decide who is still a member, and who is not. In other words, we need to be able to detect when a member has failed.

When it comes to detecting process failures, there are essentially only two mechanisms. Either processes actively send “are you alive?” messages to each other (for which they obviously expect an answer), or passively wait until messages come in from different processes. The latter approach makes sense only when it can be guaranteed that there is enough communication.

There is a huge body of theoretical work on failure detectors. What it all boils down to is that a timeout mechanism is used to check whether a process has failed. If a process P probes another process Q to see if has failed, P is said to **suspect** Q to have crashed if Q has not responded within some time.

Note 8.8 (More information: On perfect failure detectors)

It should be clear that in a synchronous distributed system, a suspected crash corresponds to a known crash. In practice, however, we will be dealing with partially synchronous systems. In that case, it makes more sense to assume **eventually perfect failure detectors**. In this case, a process P will suspect another process Q to have crashed after t time units have elapsed and still Q did not

respond to P's probe. However, if Q later does send a message that is (also) received by P, P will (1) stop suspecting Q, and (2) increase the timeout value t . Note that if Q does crash (and does not recover), P will continue to suspect Q.

In real settings, there are problems with using probes and timeouts. For example, due to unreliable networks, simply stating that a process has failed because it does not return an answer to a probe message may be wrong. In other words, it is quite easy to generate false positives. If a false positive has the effect that a perfectly healthy process is removed from a membership list, then clearly we are doing something wrong. Another serious problem is that timeouts are just plain crude. As noticed by Birman [2012], there is hardly any work on building proper failure detection subsystems that take more into account than only the lack of a reply to a single message. This statement is even more evident when looking at industry-deployed distributed systems.

There are various issues that need to be taken into account when designing a failure detection subsystem (see also Zhuang et al. [2005]). For example, failure detection can take place through gossiping in which each node regularly announces to its neighbors that it is still up and running. As we mentioned, an alternative is to let nodes actively probe each other.

Failure detection can also be done as a side-effect of regularly exchanging information with neighbors, as is the case with gossip-based information dissemination (which we discussed in Chapter 4). This approach is essentially also adopted in Obduro [Vogels, 2003]: processes periodically gossip their service availability. This information is gradually disseminated through the network. Eventually, every process will know about every other process, but more importantly, will have enough information locally available to decide whether a process has failed or not. A member for which the availability information is old, will presumably have failed.

Another important issue is that a failure detection subsystem should ideally be able to distinguish network failures from node failures. One way of dealing with this problem is not to let a single node decide whether one of its neighbors has crashed. Instead, when noticing a timeout on a probe message, a node requests other neighbors to see whether they can reach the presumed failing node. Of course, positive information can also be shared: if a node is still alive, that information can be forwarded to other interested parties (who may be detecting a link failure to the suspected node).

This brings us to another key issue: when a member failure is detected, how should other nonfaulty processes be informed? One simple, and somewhat radical approach is the following. In FUSE [Dunagan et al., 2004], processes can be joined in a group that spans a wide-area network. The group members create a spanning tree that is used for monitoring member failures. Members send ping messages to their neighbors. When a neighbor does not

respond, the pinging node immediately switches to a state in which it will also no longer respond to pings from other nodes. By recursion, it is seen that a single node failure is rapidly promoted to a group failure notification.

8.3 Reliable client-server communication

In many cases, fault tolerance in distributed systems concentrates on faulty processes. However, we also need to consider communication failures. Most of the failure models discussed previously apply equally well to communication channels. In particular, a communication channel may exhibit crash, omission, timing, and arbitrary failures. In practice, when building reliable communication channels, the focus is on masking crash and omission failures. Arbitrary failures may occur in the form of duplicate messages, resulting from the fact that in a computer network messages may be buffered for a relatively long time, and are reinjected into the network after the original sender has already issued a retransmission (see, for example, Tanenbaum and Wetherall [2010]).

Point-to-point communication

In many distributed systems, reliable point-to-point communication is established by making use of a reliable transport protocol, such as TCP. TCP masks omission failures, which occur in the form of lost messages, by using acknowledgments and retransmissions. Such failures are completely hidden from a TCP client.

However, crash failures of connections are not masked. A crash failure may occur when (for whatever reason) a TCP connection is abruptly broken so that no more messages can be transmitted through the channel. In most cases, the client is informed that the channel has crashed by raising an exception. The only way to mask such failures is to let the distributed system attempt to automatically set up a new connection, by simply resending a connection request. The underlying assumption is that the other side is still, or again, responsive to such requests.

RPC semantics in the presence of failures

Let us now take a closer look at client-server communication when using high-level communication facilities such as Remote Procedure Calls (RPCs). The goal of RPC is to hide communication by making remote procedure calls look just like local ones. With a few exceptions, so far we have come fairly close. Indeed, as long as both client and server are functioning perfectly, RPC does its job well. The problem comes about when errors occur. It is then that the differences between local and remote calls are not always easy to mask.

To structure our discussion, let us distinguish between five different classes of failures that can occur in RPC systems, as follows:

1. The client is unable to locate the server.
2. The request message from the client to the server is lost.
3. The server crashes after receiving a request.
4. The reply message from the server to the client is lost.
5. The client crashes after sending a request.

Each of these categories poses different problems and requires different solutions.

Client cannot locate the server

To start with, it can happen that the client cannot locate a suitable server. All servers might be down, for example. Alternatively, suppose that the client is compiled using a particular version of the client stub, and the binary is not used for a considerable period of time. In the meantime, the server evolves and a new version of the interface is installed; new stubs are generated and put into use. When the client is eventually run, the binder will be unable to match it up with a server and will report failure. While this mechanism is used to protect the client from accidentally trying to talk to a server that may not agree with it in terms of what parameters are required or what it is supposed to do, the problem remains of how should this failure be dealt with.

One possible solution is to have the error raise an **exception**. In some languages, (e.g., Java), programmers can write special procedures that are invoked upon specific errors, such as division by zero. In C, signal handlers can be used for this purpose. In other words, we could define a new signal type `SIGNOSERVER`, and allow it to be handled in the same way as other signals.

This approach, too, has drawbacks. To start with, not every language has exceptions or signals. Another point is that having to write an exception or signal handler destroys the transparency we have been trying to achieve. Suppose that you are a programmer and your boss tells you to write the append procedure. You smile and tell her it will be written, tested, and documented in five minutes. Then she mentions that you also have to write an exception handler as well, just in case the procedure is not there today. At this point it is pretty hard to maintain the illusion that remote procedures are no different from local ones, since writing an exception handler for “Cannot locate server” would be a rather unusual request in a nondistributed system. So much for transparency.

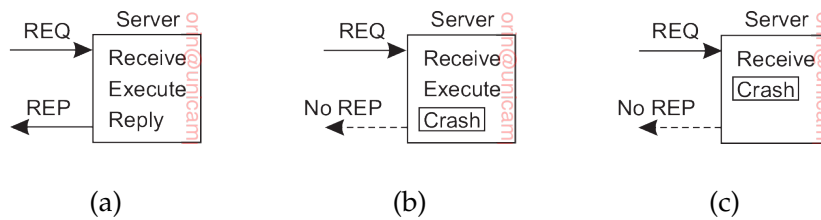


Figure 8.18: A server in client-server communication. (a) The normal case. (b) Crash after execution. (c) Crash before execution.

Lost request messages

The second item on the list is dealing with lost request messages. This is the easiest one to deal with: just have the operating system or client stub start a timer when sending the request. If the timer expires before a reply or acknowledgment comes back, the message is sent again. If the message was truly lost, the server will not be able to tell the difference between the retransmission and the original, and everything will work fine. Unless, of course, so many request messages are lost that the client gives up and falsely concludes that the server is down, in which case we are back to “Cannot locate server.” If the request was not lost, the only thing we need to do is let the server be able to detect it is dealing with a retransmission. Unfortunately, doing so is not so simple, as we explain when discussing lost replies.

Server crashes

The next failure on the list is a server crash. The normal sequence of events at a server is shown in Figure 8.18(a). A request arrives, is carried out, and a reply is sent. Now consider Figure 8.18(b). A request arrives and is carried out, just as before, but the server crashes before it can send the reply. Finally, look at Figure 8.18(c). Again a request arrives, but this time the server crashes before it can even be carried out. And, of course, no reply is sent back.

The annoying part of Figure 8.18 is that the correct treatment differs for (b) and (c). In (b) the system has to report failure back to the client (e.g., raise an exception), whereas in (c) it can just retransmit the request. The problem is that the client’s operating system cannot tell which is which. All it knows is that its timer has expired.

Three schools of thought exist on what to do here [Spector, 1982]. One philosophy is to wait until the server reboots (or let the client’s middleware transparently rebind to a new server) and try the operation again. The idea is to keep trying until a reply has been received, then give it to the client. This technique is called **at-least-once semantics** and guarantees that the RPC has been carried out at least one time, but possibly more.

The second philosophy gives up immediately and reports back failure. This approach is called **at-most-once semantics** and guarantees that the RPC has been carried out at most one time, but possibly not at all.

The third philosophy is to guarantee nothing. When a server crashes, the client gets no help and no promises about what happened. The RPC may have been carried out anywhere from zero to a large number of times. The main virtue of this scheme is that it is easy to implement.

None of these are terribly attractive. What one would like is **exactly-once semantics**, but in general, there is no way to arrange this. Imagine that the remote operation consists of processing a document such as producing a number of PDF files from L^AT_EX and other sources. The server sends a completion message to the client when the document has been completely processed. Also assume that when a client issues a request, it receives an acknowledgment that the request has been delivered to the server. There are two strategies the server can follow. It can either send a completion message just before it actually tells the document processor to do its work, or after the document has been processed.

Assume that the server crashes and subsequently recovers. It announces to all clients that it has just crashed but is now up and running again. The problem is that the client does not know whether its request to process a document will actually have been carried out.

There are four strategies the client can follow. First, the client can decide to *never* reissue a request, at the risk that the document will not be processed. Second, it can decide to *always* reissue a request, but this may lead to the document being processed twice (which may easily incur a significant amount of work when dealing with intricate documents). Third, it can decide to reissue a request only if it did not yet receive an acknowledgment that its request had been delivered to the server. In that case, the client is counting on the fact that the server crashed before the request could be delivered. The fourth and last strategy is to reissue a request only if it has received an acknowledgment for the request. With two strategies for the server, and four for the client, there are a total of eight combinations to consider. Unfortunately, as it turns out, no combination is satisfactory: it can be shown that for any combination either the request is lost forever, or carried out twice.

Note 8.9 (Advanced: Why fully transparent server recovery is impossible)

To explain the situation of server recovery, note that there are three events that can happen at the server: send the completion message (M), complete the processing of the document (P), and crash (C). Note that crashing *during* the processing of a document is considered the same as crashing before its completion. These events can occur in six different orderings:

1. $M \rightarrow P \rightarrow C$: A crash occurs after sending the completion message and processing the document.

2. $M \rightarrow C(\rightarrow P)$: A crash happens after sending the completion message, but before the document could be (completely) processed.
3. $P \rightarrow M \rightarrow C$: A crash occurs after sending the completion message and processing the document.
4. $P \rightarrow C(\rightarrow M)$: The document was processed, after which a crash occurs before the completion message could be sent.
5. $C(\rightarrow P \rightarrow M)$: A crash happens before the server could complete the processing of the document.
6. $C(\rightarrow M \rightarrow P)$: A crash happens before the server could even do anything.

The parentheses indicate an event that can no longer happen because the server already crashed. Figure 8.19 shows all possible combinations. As can be readily verified, there is no combination of client strategy and server strategy that will work correctly under all possible event sequences. The bottom line is that the client can never know whether the server crashed just before or after having the text printed.

Reissue strategy	Strategy $M \rightarrow P$			Strategy $P \rightarrow M$		
	MPC	MC(P)	C(MP)	PMC	PC(M)	C(PM)
Always	DUP	OK	OK	DUP	DUP	OK
Never	OK	ZERO	ZERO	OK	OK	ZERO
Only when ACKed	DUP	OK	ZERO	DUP	OK	ZERO
Only when not ACKed	OK	ZERO	OK	OK	DUP	OK
Client	Server			Server		
	OK	= Document processed once				
	DUP	= Document processed twice				
	ZERO	= Document not processed at all				

Figure 8.19: Different combinations of client and server strategies in the presence of server crashes. Events between brackets never take place because of a previous crash.

In short, the possibility of server crashes radically changes the nature of RPC and clearly distinguishes single-processor systems from distributed ones. In the former case, a server crash also implies a client crash, so recovery is neither possible nor necessary. In the latter we can and should take action.

Lost reply messages

Lost replies can also be difficult to deal with. The obvious solution is just to rely on a timer again that has been set by the client's operating system. If no reply is forthcoming within a reasonable period, just send the request once more. The trouble with this solution is that the client is not really sure why there was no answer. Did the request or reply get lost, or is the server merely slow? It may make a difference.

In particular, some operations can safely be repeated as often as necessary with no damage being done. A request such as asking for the first 1024 bytes of a file has no side effects and can be executed as often as necessary

without any harm being done. A request that has this property is said to be **idempotent**.

Now consider a request to a banking server asking to transfer money from one account to another. If the request arrives and is carried out, but the reply is lost, the client will not know this and will retransmit the message. The bank server will interpret this request as a new one, and will carry it out too. Twice the amount of money will be transferred. Transferring money is not idempotent.

One way of solving this problem is to try to structure all the requests in an idempotent way. In practice, however, many requests (e.g., transferring money) are inherently nonidempotent, so something else is needed. Another method is to have the client assign each request a sequence number. By having the server keep track of the most recently received sequence number from each client that is using it, the server can tell the difference between an original request and a retransmission and can refuse to carry out any request a second time. However, the server will still have to send a response to the client. Note that this approach does require that the server maintains administration on each client. Furthermore, it is not clear how long to maintain this administration. An additional safeguard is to have a bit in the message header that is used to distinguish initial requests from retransmissions (the idea being that it is always safe to perform an original request; retransmissions may require more care).

Client crashes

The final item on the list of failures is the client crash. What happens if a client sends a request to a server to do some work and crashes before the server replies? At this point a computation is active and no parent is waiting for the result. Such an unwanted computation is called an **orphan (computation)**.

Orphan computations can cause a variety of problems that can interfere with normal operation of the system. As a bare minimum, they waste processing power. They can also lock files or otherwise tie up valuable resources. Finally, if the client reboots and does the RPC again, but the reply from the orphan comes back immediately afterward, confusion can result.

What can be done about orphans? Four solutions have been proposed [Nelson, 1981]. First, before a client stub sends an RPC message, it makes a log entry telling what it is about to do. The log is kept on disk or some other medium that survives crashes. After a reboot, the log is checked and the orphan is explicitly killed off. This solution is called **orphan extermination**.

The disadvantage of this scheme is the horrendous expense of writing a disk record for every RPC. Furthermore, it may not even work, since orphans themselves may do RPCs, thus creating **grandorphans** or further descendants that are difficult or impossible to locate. Finally, the network may be partitioned, for example, due to a failed gateway, making it impossible

to kill them even if they can be located. All in all, this is not a promising approach.

With the second solution, called **reincarnation**, all these problems can be solved without the need to write disk records. The way it works is to divide time up into sequentially numbered epochs. When a client reboots, it broadcasts a message to all machines declaring the start of a new epoch. When such a broadcast comes in, all remote computations are killed. Of course, if the network is partitioned, some orphans may survive. Fortunately, however, when they report back, their replies will contain an obsolete epoch number, making them easy to detect.

The third solution is a variant on this idea, but somewhat less draconian. It is called **gentle reincarnation**. When an epoch broadcast comes in, each machine checks to see if it has any remote computations running locally, and if so, tries its best to locate their owners. Only if the owners cannot be located anywhere is the computation killed.

In the fourth solution, called **expiration**, each RPC is given a standard amount of time, T , to do the job. If it cannot finish, it must explicitly ask for another quantum. Of course, this is quite a nuisance. On the other hand, if after a crash the client waits a time T before rebooting, all orphans are sure to be gone. The problem to be solved here is choosing a reasonable value of T in the face of RPCs with wildly differing requirements.

In practice, all of these methods are crude and undesirable. Worse yet, killing an orphan may have unforeseen consequences. For example, suppose that an orphan has obtained locks on one or more files or database records. If the orphan is suddenly killed, these locks may remain forever. Also, an orphan may have already made entries in various remote queues to start up other processes at some future time, so even killing the orphan may not remove all traces of it. Conceivably, it may even have started again, with unforeseen consequences. Orphan elimination is discussed in more detail by Panziera and Shrivastava [1988].

8.4 Reliable group communication

Considering how important process resilience by replication is, it is not surprising that reliable multicast services are important as well. Such services guarantee that messages are delivered to all members in a process group. Unfortunately, reliable multicasting turns out to be surprisingly tricky. In this section, we take a closer look at the issues involved in reliably delivering messages to a process group. Let us first define what reliable group communication actually is. Intuitively, it means that a message that is sent to a process group should be delivered to each member of that group. If we separate the logic of handling messages from the core functionality of a group member, we can conveniently make the distinction between *receiving* messages and

delivering messages, as illustrated in Figure 8.20. A message is received by a message-handling component, which, in turn, delivers a message to the component containing the core functionality of a group member. Informally, a message that is received by process P will also be delivered by P.

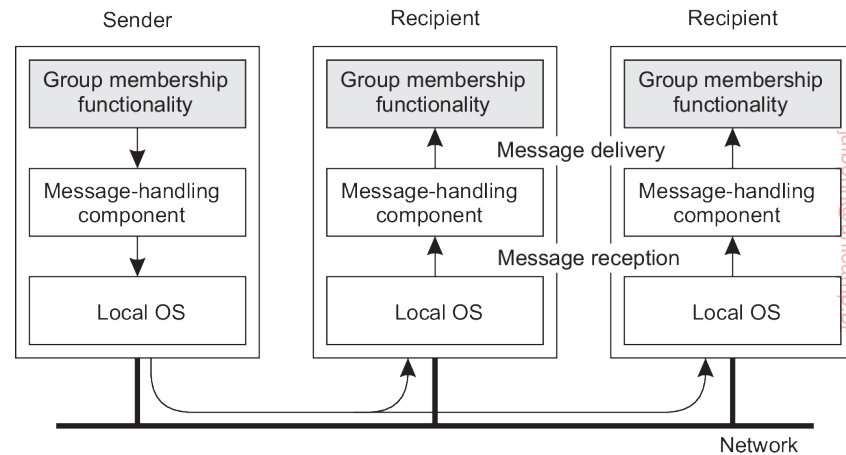


Figure 8.20: The distinction between receiving and delivering messages.

As an example, ensuring that messages from the same sender are delivered in the same order as they were sent, is typically taken care of by a message-handling component. Likewise, providing reliable message-passing is a feature that can and should be separated from the core functionality of a group member, and is typically implemented by a message-handling component (if not by the underlying operating system).

With this separation between receiving and delivering messages, we can be more precise about what reliable group communication means. Let us make a distinction between reliable communication in the presence of faulty processes, and reliable communication when processes are assumed to operate correctly. In the first case, group communication is considered to be reliable when it can be guaranteed that a message is received and subsequently delivered by all nonfaulty group members.

The tricky part here is that agreement should be reached on what the group actually looks like before a message can be delivered. If a sender intended to have a message delivered by each member of a group G , but that, for whatever reason, at the time of delivery we actually have another group $G' \neq G$, we should ask ourselves if the message can be delivered or not.

The situation becomes simpler if we can ignore consensus on group membership. In particular, let us first assume that a sending process has a list of intended recipients. In that case, it can simply deploy reliable transport-level protocols such as TCP and, one by one, sends its message to each recipient. If a receiving process fails, the message may be resent later when the process

recovers, or ignored altogether (for example, because the sender had left the group). In case a group member is expected to send a response, even if it is just an acknowledgement, communication can be speeded up by separating the sending of a request from receiving a response, as illustrated by the message sequence charts in Figure 8.21.

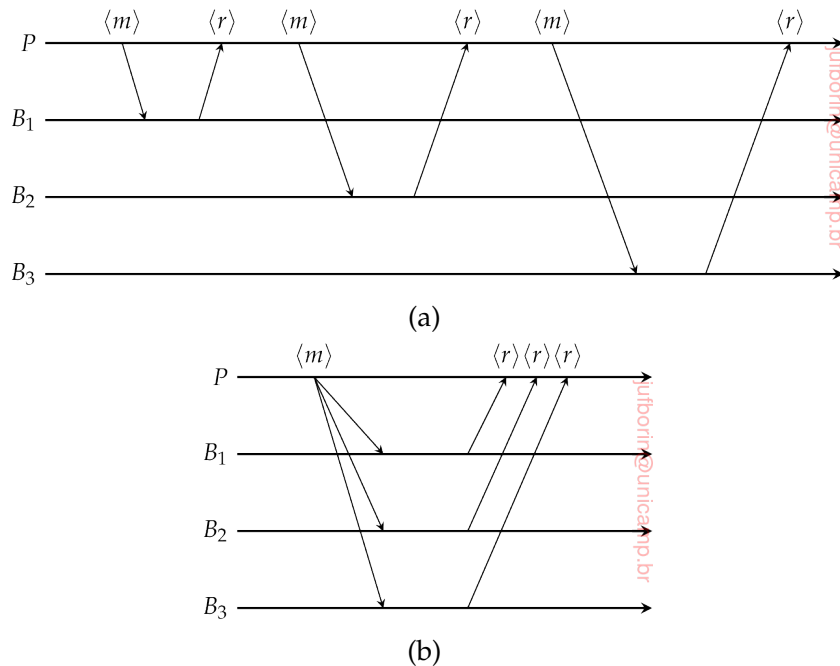


Figure 8.21: (a) A sender sends out requests, but waits for a response before sending out the next one. (b) Requests are sent out in parallel, after which the sender waits for incoming responses.

Most transport layers offer reliable point-to-point channels; they rarely offer reliable communication to a group of processes. The best they offer is to let a process set up a point-to-point connection to each other process it wants to communicate with. When process groups are relatively small, this approach to establishing reliability is a straightforward and practical solution. On the other hand, we can often assume that the underlying communication system does offer unreliable multicasting, meaning that a multicast message may be lost part way and delivered by some, but not all, of the intended receivers.

A simple solution to reach reliable group communication is shown in Figure 8.22. The sending process assigns a sequence number to each message it multicasts and stores the message locally in a history buffer. Assuming the receivers are known to the sender, the sender simply keeps the message in its history buffer until each receiver has returned an acknowledgment. A receiver can suspect it is missing a message m with sequence number s

when it has received messages with sequence numbers higher than s . In that case, it returns a negative acknowledgement to the sender, requesting for a retransmission of m .

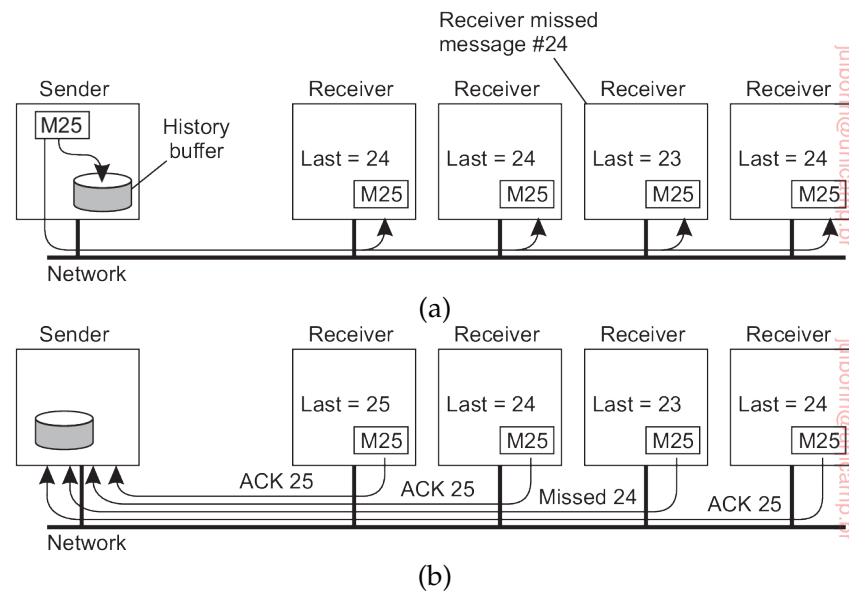


Figure 8.22: A solution for reliable multicasting. (a) Message transmission. (b) Reporting feedback.

There are various design trade-offs to be made. For example, to reduce the number of messages returned to the sender, acknowledgments could possibly be piggybacked with other messages. Also, retransmitting a message can be done using point-to-point communication to each requesting process, or using a single multicast message sent to all processes. General issues on reliable multicasting are discussed by Popescu et al. [2007]. A survey and overview of reliable multicasting in the context of publish/subscribe systems, which is also relevant here is [Esposito et al., 2013].

Note 8.10 (Advanced: Scalability in reliable multicasting)

The main problem with the reliable multicast scheme just described is that it cannot support large numbers of receivers. If there are N receivers, the sender must be prepared to accept at least N acknowledgments. With many receivers, the sender may be swamped with such feedback messages, which is also referred to as a **feedback implosion**. When replicating processes for fault tolerance, this situation is not likely to occur as process groups are relatively small. When replicating for performance, we have a different case. Moreover, we may then also need to take into account that the receivers are spread across a wide-area network.

One solution to the problem of a feedback implosion is not to have receivers acknowledge the receipt of a message. Instead, a receiver returns a feedback message only to inform the sender it is missing a message. Returning only such negative acknowledgments can be shown to generally scale better [Towsley et al., 1997], but no hard guarantees can be given that feedback implosions will never happen.

Another problem with returning only negative acknowledgments is that the sender will, in theory, be forced to keep a message in its history buffer forever. Because the sender can never know if a message has been correctly delivered to all receivers, it should always be prepared for a receiver requesting the retransmission of an old message. In practice, the sender will remove a message from its history buffer after some time has elapsed to prevent the buffer from overflowing. However, removing a message is done at the risk of a request for a retransmission not being honored.

Several proposals for scalable reliable multicasting exist. A comparison between different schemes can be found in [Levine and Garcia-Luna-Aceves, 1998]. We now briefly discuss two very different approaches that are representative of many existing solutions.

Nonhierarchical feedback control The key issue to scalable solutions for reliable multicasting is to reduce the number of feedback messages that are returned to the sender. A popular model that has been applied to several wide-area applications is **feedback suppression**. This scheme underlies the **Scalable Reliable Multicasting (SRM)** protocol developed by Floyd et al. [1997] and works as follows.

First, in SRM, receivers never acknowledge the successful delivery of a multicast message, but instead, report only when they are missing a message. How message loss is detected is left to the application. Only negative acknowledgments are returned as feedback. Whenever a receiver notices that it missed a message, it *multicasts* its feedback to the rest of the group.

Multicasting feedback allows another group member to suppress its own feedback. Suppose several receivers missed message m . Each of them will need to return a negative acknowledgment to the sender, S , so that m can be retransmitted. However, if we assume that retransmissions are always multicast to the entire group, it is sufficient that only a single request for retransmission reaches S .

For this reason, a receiver R that did not receive message m schedules a feedback message with some random delay. That is, the request for retransmission is not sent until some random time has elapsed. If, in the meantime, another request for retransmission for m reaches R , R will suppress its own feedback, knowing that m will be retransmitted shortly. In this way, ideally, only a single feedback message will reach S , which in turn subsequently retransmits m . This scheme is shown in Figure 8.23.

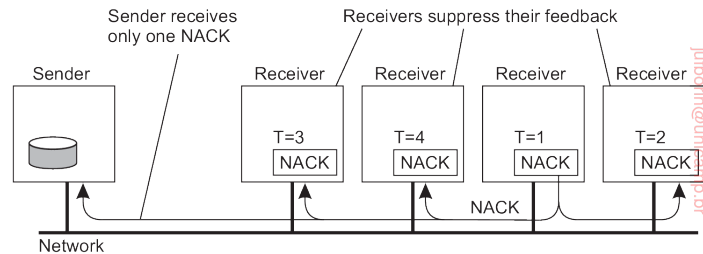


Figure 8.23: Several receivers have scheduled a request for retransmission, but the first retransmission request leads to the suppression of others.

Feedback suppression has shown to scale reasonably well, and has been used as the underlying mechanism for a number of collaborative Internet applications, such as a shared whiteboard. However, the approach also introduces a number of serious problems. First, ensuring that only one request for retransmission is returned to the sender requires a reasonably accurate scheduling of feedback messages at each receiver. Otherwise, many receivers will still return their feedback at the same time. Setting timers accordingly in a group of processes that is dispersed across a wide-area network is not that easy.

Another problem is that multicasting feedback also interrupts those processes to which the message has been successfully delivered. In other words, other receivers are forced to receive and process messages that are useless to them. The only solution to this problem is to let receivers that have not received message m join a separate multicast group for m , as explained by Kasera et al. [1997]. Unfortunately, this solution requires that groups can be managed in a highly efficient manner, which is hard to accomplish in a wide-area system. A better approach is therefore to let receivers that tend to miss the same messages team up and share the same multicast channel for feedback messages and retransmissions. Details on this approach are found in [Liu et al., 1998].

To enhance the scalability of SRM, it is useful to let receivers assist in local recovery. In particular, if a receiver to which message m has been successfully delivered, receives a request for retransmission, it can decide to multicast m even before the retransmission request reaches the original sender. Further details can be found in [Floyd et al., 1997] and [Liu et al., 1998].

Hierarchical feedback control Feedback suppression as just described is basically a nonhierarchical solution. However, achieving scalability for very large groups of receivers requires that hierarchical approaches are adopted. A solution is shown in Figure 8.24. The group of receivers is partitioned into a number of subgroups, which are subsequently organized into a tree. Within each subgroup, any reliable multicasting scheme that works for small groups can be used. Each subgroup appoints a local coordinator, which represents that group in the multicast tree. A link in the tree between two nodes corresponds to a reliable connection between the coordinators of the respective subgroups.

When a process S in group G wants to send a message, it simply uses the reliable multicast scheme for G to reach all its members, including the group's coordinator, say C . C , in turn, will forward the message to its neighboring coordinators. As a general rule, a coordinator will forward an incoming message m to all its neighboring coordinators, except the one from which it received m . In addition, a coordinator will reliably multicast the incoming message to all members of the subgroup it represents, and notably also handle retransmissions for that group.

In an ACK-based scheme, if coordinator C of group G sends a message m to coordinator C' of another, neighboring group G' , it will keep m in its history buffer at least until C' has sent an acknowledgement. In a NACK-based scheme, only if G' detects it has missed m (and thus also all members of G' , and all coordinators to which G' would have forwarded m), it will send a NACK message to C . It is thus seen that a single ACK or NACK message from a coordinator, aggregates many feedback control messages from other processes, leading to a much more scalable reliable multicasting scheme. Scalability is further improved by letting a coordinator handle the retransmissions to neighboring coordinators to which it had forwarded a message.

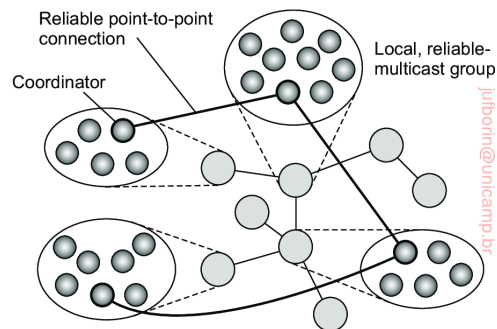


Figure 8.24: The essence of hierarchical reliable multicasting. Each local coordinator forwards the message to its neighboring coordinators in the tree and later handles retransmission requests.

Note that the nonhierarchical feedback control which we discussed before can be used to improve the scalability of a single multicast group. Together with hierarchical feedback control, we would combine relatively large reliable-multicast subgroups into potentially large trees, thus being able to support reliable multicasting for very large groups of processes.

The main problem with hierarchical solutions is the construction and management of the tree: how are subgroups formed, which processes are appointed to be coordinator, and how are the subgroups organized in a tree. In many cases, a tree needs to be constructed dynamically. Unfortunately, traditional network-level solutions provide almost no adequate services for tree management. For this reason, application-level multicasting solutions as we discussed in Section 4.4 have gained popularity.

Gossip-based scalable reliable multicasting Finally, let us briefly consider gossip-based multicasting schemes, in particular the following push-pull **anti-entropy** scheme that we discussed extensively in Section 4.4 .

In this scheme, a node P picks another node Q at random, and subsequently exchanges updates with Q. In other words, P pushes updates that Q has not seen before to Q, and pulls in any updates that Q has, but which were missed by P. After the exchange, both processes have the same data. Clearly, this scheme is already inherently robust, for if the communication between P and Q fails for whatever reason, P will simply pick some other node to exchange updates. The net effect is that the speed by which an update propagates through the system slows down, but the reliability is affected only in extreme cases. Nevertheless, this slowdown is considered important for some applications. In this light, the comparison between traditional tree-based multicasting and gossip-based multicasting for the purpose of aggregation as discussed by Nyers and Jelasity [2015] may be of interest.

In conclusion, building reliable multicast schemes that can scale to a large number of receivers spread across large networks, is a difficult problem. No single best solution exists, and each solution introduces new problems.

Atomic multicast

Let us now return to the situation in which we need to achieve reliable multicasting in the presence of process failures. In particular, what is often needed in a distributed system is the guarantee that a message is delivered to either all (nonfaulty) group members or to none at all. This is also known as the **atomic multicast problem**, to which we return with greater precision below.

To see why atomicity is so important, consider a replicated database constructed as an application on top of a distributed system. The distributed system offers reliable multicasting facilities. In particular, it allows the construction of process groups to which messages can be reliably sent. The replicated database is therefore constructed as a group of processes, one process for each replica. Update operations are always multicast to all replicas and subsequently performed locally. We are thus assuming that an active-replication protocol is being used.

To keep matters simple, assume a client contacts a replica P and requests it to perform an update. The replica does so by multicasting the update to the other group members. Unfortunately, before the multicast completes, P crashes, leaving the rest of the group in a difficult position: some group members will have received the update request; others will not. If the members who have received the request deliver it to the database, then obviously we will have an inconsistent replicated database. Some replicas will have processed the update, others will not. This situation needs to be avoided and we should

either have that the update is delivered to all nonfaulty members, or to none at all. The former case reflects that *P* crashed *after* completing the multicast, while the latter represents *P* crashing before it even got a chance to request the update.

Both these situations are fine, and correspond to the case in which a client communicates with a single server that is allowed to crash. If a number of the group members would execute the update, while others would not, distribution transparency is at stake, but even worse, the client would not know what to make of the situation.

Virtual synchrony

Reliable multicast in the presence of process failures can be accurately defined in terms of process groups and changes to group membership. As we did earlier, we make a distinction between *receiving* and *delivering* a message. In particular, we again adopt a model in which the distributed system consists of message-handling components as was shown in Figure 8.20. A received message is locally buffered in this component until it can be delivered to the application, which is logically placed as a group member at a higher layer.

The whole idea of atomic multicasting is that a multicast message *m* is uniquely associated with a list of processes that should deliver it. This delivery list corresponds to a **group view**, namely, the view on the set of processes contained in the group, which the sender had at the time message *m* was multicast. An important observation is that each process on that list has the same view. In other words, they should all agree that *m* should be delivered by each one of them and by no other process.

Now suppose that message *m* is multicast at the time its sender, say *P*, has group view *G*. Furthermore, assume that while the multicast is taking place, another process *Q* joins or leaves the group. This change in group membership is naturally announced to all processes in *G*. Stated somewhat differently, a **view change** takes place by multicasting a message *vc* announcing the joining or leaving of *Q*. We now have two multicast messages simultaneously in transit: *m* and *vc*. What we need to guarantee is that *m* is either delivered by all processes in *G* before any one executes the view change as specified by *vc*, or *m* is not delivered at all. Note that this requirement is comparable to total-ordered multicasting, which we discussed in Chapter 6.

A question that quickly comes to mind is that if *m* is not delivered by any process, how can we speak of a *reliable* multicast protocol? In principle, there is only one case in which delivery of *m* is allowed to fail: when the group membership change is the result of the sender *P* of *m* crashing. In that case, either all remaining (nonfaulty) members of *G* should deliver *m* before agreeing *P* is no longer member of the group, or none should deliver *m*. As mentioned before, the latter corresponds to the situation that *P* is considered to have crashed before it had a chance to send *m*.

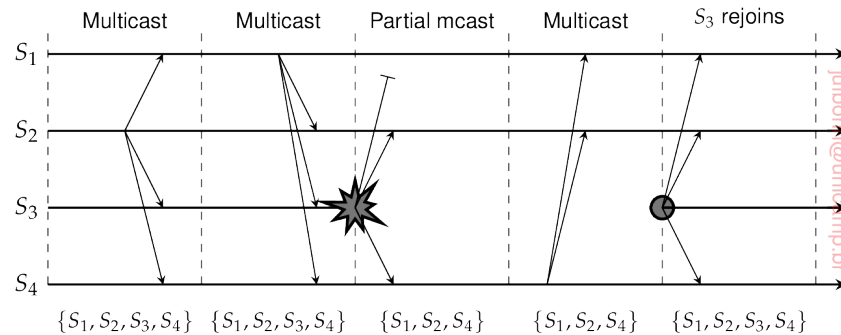


Figure 8.25: The principle of virtual synchronous multicast.

This stronger form of reliable multicast guarantees that a message multicast to group view G is delivered by each nonfaulty process in G . If the sender of the message crashes during the multicast, the message is either delivered to all remaining processes, or ignored by each of them. Such a reliable multicast is said to be **virtually synchronous** [Birman and Joseph, 1987].

To illustrate these matters, consider the four processes shown in Figure 8.25. At a certain point in time, we have a group consisting of S_1 , S_2 , S_3 , and S_4 . After some messages have been multicast, S_3 crashes. However, before crashing, it succeeded in multicasting a message to processes S_2 and S_4 , but not to S_1 . However, virtual synchrony in this case guarantees that the message is not delivered at all, effectively establishing the situation that the message was never sent before S_3 crashed.

After S_3 has been removed from the group, communication proceeds between the remaining group members. Later, when S_3 recovers, it can join the group again, after its state has been brought up to date.

The principle of virtual synchrony comes from the fact that all multicasts take place between view changes. Put somewhat differently, a view change acts as a barrier across which no multicast can pass. In a sense, it is comparable to the use of a synchronization variable in distributed data stores as discussed in the previous chapter. All multicasts that are in transit while a view change takes place are completed before the view change comes into effect. The implementation of virtual synchrony is not trivial as we discuss below.

Message Ordering

Virtual synchrony allows an application developer to think about multicasts as taking place in epochs that are separated by group membership changes. However, nothing has yet been said concerning the ordering of multicasts. In general, four different orderings are distinguished:

1. Unordered multicasts

2. FIFO-ordered multicasts
3. Causally ordered multicasts
4. Totally ordered multicasts

A **reliable, unordered multicast** is a virtually synchronous multicast in which no guarantees are given concerning the order in which received messages are delivered by different processes. To explain, assume that reliable multicasting is supported by a library providing a send and a receive primitive. The receive operation blocks the caller until a message can be delivered.

Event order	Process P_1	Process P_2	Process P_3
1	sends m_1	receives m_1	receives m_2
2	sends m_2	receives m_2	receives m_1

Figure 8.26: Three communicating processes in the same group. The ordering of events per process is shown along the vertical axis.

Now suppose a sender P_1 multicasts two messages to a group while two other processes in that group are waiting for messages to arrive, as shown in Figure 8.26. Assuming that processes do not crash or leave the group during these multicasts, it is possible that the message-handling component at P_2 first receives message m_1 and then m_2 . Because there are no message-ordering constraints, the messages may be delivered in the order that they are received. In contrast, the message-handling component at P_3 may first receive message m_2 followed by m_1 , and delivers these two in this same order to the higher-level application of P_3 .

In the case of **reliable FIFO-ordered multicasts**, the message-handling component layer is forced to deliver incoming messages from the same process in the same order as they have been sent. Consider the communication within a group of four processes, as shown in Figure 8.27. With FIFO ordering, the only thing that matters is that message m_1 is always delivered before m_2 , and, likewise, that message m_3 is always delivered before m_4 . This rule has to be obeyed by all processes in the group. In other words, when the communication layer at P_3 receives m_2 first, it will wait with delivery to P_3 until it has received and delivered m_1 .

However, there is no constraint regarding the delivery of messages sent by different processes. In other words, if process P_2 receives m_1 before m_3 , it may deliver the two messages in that order. Meanwhile, process P_3 may have received m_3 before receiving m_1 . FIFO ordering states that P_3 may deliver m_3 before m_1 , although this delivery order is different from that of P_2 .

Finally, **reliable causally ordered multicast** delivers messages so that potential causality between different messages is preserved. In other words, if a message m_1 causally precedes another message m_2 , regardless of whether

Event order	Process P ₁	Process P ₂	Process P ₃	Process P ₄
1	sends m ₁	receives m ₁	receives m ₃	sends m ₃
2	sends m ₂	receives m ₃	receives m ₁	sends m ₄
3		receives m ₂	receives m ₂	
4		receives m ₄	receives m ₄	

Figure 8.27: Four processes in the same group with two different senders, and a possible delivery order of messages under FIFO-ordered multicasting.

they were multicast by the same sender, then the communication layer at each receiver will always deliver m_2 after it has received and delivered m_1 . Note that causally ordered multicasts can be implemented using vector timestamps as discussed in Chapter 6.

Besides these three orderings, there may be the additional constraint that message delivery is to be totally ordered as well. **Total-ordered delivery** means that regardless of whether message delivery is unordered, FIFO ordered, or causally ordered, it is required additionally that when messages are delivered, they are delivered in the same order to all group members.

For example, with the combination of FIFO and totally ordered multicast, processes P_2 and P_3 in Figure 8.27 may both first deliver message m_3 and then message m_1 . However, if P_2 delivers m_1 before m_3 , while P_3 delivers m_3 before delivering m_1 , they would violate the total-ordering constraint. Note that FIFO ordering should still be respected. In other words, m_2 should be delivered after m_1 and, accordingly, m_4 should be delivered after m_3 .

Virtually synchronous reliable multicasting offering total-ordered delivery of messages is called **atomic multicasting**. It encompasses the all-or-nothing property discussed before, yet the full definition also requires a total-ordered delivery of messages. With the three different message ordering constraints discussed above, this leads to six forms of reliable multicasting as shown in Figure 8.28 [Hadzilacos and Toueg, 1993].

Multicast	Basic message ordering	TO delivery?
Reliable multicast	None	No
FIFO multicast	FIFO-ordered delivery	No
Causal multicast	Causal-ordered delivery	No
Atomic multicast	None	Yes
FIFO atomic multicast	FIFO-ordered delivery	Yes
Causal atomic multicast	Causal-ordered delivery	Yes

Figure 8.28: Six different versions of virtually synchronous reliable multicasting and considering totally ordered delivery.

Note 8.11 (Advanced: Implementing virtual synchrony)

Let us now consider a possible implementation of a virtually synchronous reliable multicast. An example of such an implementation appears in Isis, a fault-tolerant distributed system that has been in practical use in industry for several years. We will focus on some of the implementation issues of this technique as described in Birman et al. [1991].

Reliable multicasting in Isis makes use of available reliable point-to-point communication facilities of the underlying network, in particular, TCP. Multicasting a message m to a group of processes is implemented by reliably sending m to each group member. As a consequence, although each transmission is guaranteed to succeed, there are no guarantees that *all* group members receive m . In particular, the sender may fail before having transmitted m to each member.

Besides reliable point-to-point communication, Isis also assumes that messages from the same source are received by a communication layer in the order they were sent by that source. In practice, this requirement is solved by using TCP connections for point-to-point communication.

The main problem that needs to be solved is to guarantee that all messages sent to view G are delivered to all nonfaulty processes in G before the next group membership change takes place. The first issue that needs to be taken care of is making sure that each process in G has received all messages that were sent to G . Note that because the sender of a message m to G may have failed before completing its multicast, there may indeed be processes in G that will never receive m . Because the sender has crashed, these processes should get m from somewhere else.

The solution to this problem is to let every process in G keep m until it knows for sure that all members in G have received it. If m has been received by all members in G , m is said to be **stable**. Only stable messages are allowed to be delivered. To ensure stability, it is sufficient to select an arbitrary live process in G and request it to send m to all other processes in G .

To be more specific, assume the current view is G_i , but that it is necessary to install the next view G_{i+1} . Without loss of generality, we assume that G_i and G_{i+1} differ by at most one process. A process P notices the view change when it receives a view-change message. Such a message may come from the process wanting to join or leave the group, or from a process that had detected the failure of a process in G_i that is now to be removed, as shown in Figure 8.29(a).

When a process P receives the view-change message for G_{i+1} , it first forwards a copy of any unstable message from G_i it still has to every process in G_{i+1} , and subsequently marks it as being stable. Recall that Isis assumes point-to-point communication is reliable, so that forwarded messages are never lost. Such forwarding guarantees that all messages in G_i that have been received by at least one process are received by all nonfaulty processes in G_i . Note that it would also have been sufficient to elect a single coordinator to forward unstable messages.

To indicate that P no longer has any unstable messages and that it is prepared to install G_{i+1} as soon as the other processes can do that as well, it multicasts a **flush message** for G_{i+1} , as shown in Figure 8.29(b). After P has received a flush

message for G_{i+1} from each other process, it can safely install the new view, as shown in Figure 8.29(c).

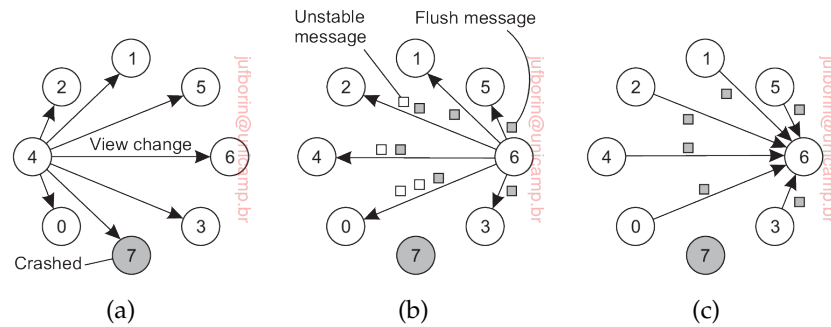


Figure 8.29: (a) Process 4 notices that process 7 has crashed and sends a view change. (b) Process 6 sends out all its unstable messages, followed by a flush message. (c) Process 6 installs the new view when it has received a flush message from everyone else.

When a process Q receives a message m while Q still believes the current view is G_i , it delivers m taking any additional message-ordering constraints into account. If it had already received m , it considers the message to be a duplicate and discards it.

Because process Q will eventually receive the view-change message for G_{i+1} , it will also first forward any of its unstable messages and subsequently wrap things up by sending a flush message for G_{i+1} . Note that due to the assumed FIFO-message ordering as provided by the underlying communication layer, a flush message from a process is always received after the receipt of an unstable message from that same process.

The major flaw in the protocol described so far is that it cannot deal with process failures while a new view change is being announced. In particular, it assumes that until the new view G_{i+1} has been installed by each member in G_{i+1} , no process in G_{i+1} will fail (which would lead to a next view G_{i+2}). This problem is solved by announcing view changes for any view G_{i+k} even while previous changes have not yet been installed by all processes. The details are rather intricate, yet the principle should be clear.

8.5 Distributed commit

The atomic multicasting problem discussed in the previous section is an example of a more general problem, known as **distributed commit**. The distributed commit problem involves having an operation being performed by each member of a process group, or none at all. In the case of reliable multicasting, the operation is the delivery of a message. With distributed transactions, the operation may be the commit of a transaction at a single site

that takes part in the transaction. Other examples of distributed commit, and how it can be solved are discussed by Tanisch [2000].

Distributed commit is often established by means of a coordinator. In a simple scheme, this coordinator tells all other processes that are also involved, called participants, whether or not to (locally) perform the operation in question. This scheme is referred to as a **one-phase commit protocol**. It has the obvious drawback that if one of the participants cannot actually perform the operation, there is no way to tell the coordinator. For example, in the case of distributed transactions, a local commit may not be possible because this would violate concurrency control constraints.

In practice, more sophisticated schemes are needed, the most common one being the two-phase commit protocol, which we discuss in detail below. The main drawback of this protocol is that it cannot generally efficiently handle the failure of the coordinator. To that end, a three-phase protocol has been developed, which we discuss separately in Note 8.13.

The original **two-phase commit protocol (2PC)** is due to Gray [1978]. Without loss of generality, consider a distributed transaction involving the participation of a number of processes each running on a different machine. Assuming that no failures occur, the protocol consists of the following two phases, each consisting of two steps (see also Bernstein and Newcomer [2009]):

1. The coordinator sends a **VOTE-REQUEST** message to all participants.
2. When a participant receives a **VOTE-REQUEST** message, it returns either a **VOTE-COMMIT** message to the coordinator telling the coordinator that it is prepared to locally commit its part of the transaction, or otherwise a **VOTE-ABORT** message.
3. The coordinator collects all votes from the participants. If all participants have voted to commit the transaction, then so will the coordinator. In that case, it sends a **GLOBAL-COMMIT** message to all participants. However, if one participant had voted to abort the transaction, the coordinator will also decide to abort the transaction and multicasts a **GLOBAL-ABORT** message.
4. Each participant that voted for a commit waits for the final reaction by the coordinator. If a participant receives a **GLOBAL-COMMIT** message, it locally commits the transaction. Otherwise, when receiving a **GLOBAL-ABORT** message, the transaction is locally aborted as well.

The first phase is the voting phase, and consists of steps 1 and 2. The second phase is the decision phase, and consists of steps 3 and 4. These four steps are shown as finite state diagrams in Figure 8.30.

Several problems arise when this basic 2PC protocol is used in a system where failures occur. First, note that the coordinator as well as the participants have states in which they block waiting for incoming messages. Consequently, the protocol can easily fail when a process crashes for other processes may be

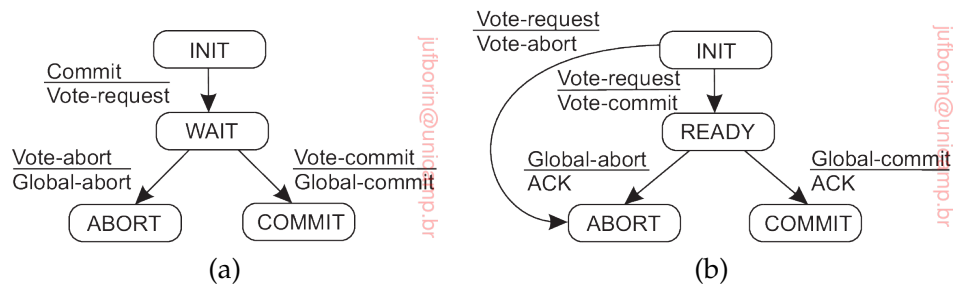


Figure 8.30: (a) The finite state machine for the coordinator in 2PC. (b) The finite state machine for a participant.

indefinitely waiting for a message from that process. For this reason, timeout mechanisms are used. These mechanisms are explained in the following pages.

When taking a look at the finite state machines in Figure 8.30, it can be seen that there are a total of three states in which either a coordinator or participant is blocked waiting for an incoming message. First, a participant may be waiting in its INIT state for a VOTE-REQUEST message from the coordinator. If that message is not received after some time, the participant will simply decide to locally abort the transaction, and thus send a VOTE-ABORT message to the coordinator.

Likewise, the coordinator can be blocked in state WAIT, waiting for the votes of each participant. If not all votes have been collected after a certain period of time, the coordinator should vote for an abort as well, and subsequently send GLOBAL-ABORT to all participants.

Finally, a participant can be blocked in state READY, waiting for the global vote as sent by the coordinator. If that message is not received within a given time, the participant cannot simply decide to abort the transaction. Instead, it must find out which message the coordinator actually sent. The simplest solution to this problem is to let each participant block until the coordinator recovers again.

A better solution is to let a participant P contact another participant Q to see if it can decide from Q's current state what it should do. For example, suppose that Q had reached state COMMIT. This is possible only if the coordinator had sent a GLOBAL-COMMIT message to Q just before crashing. Apparently, this message had not yet been sent to P. Consequently, P may now also decide to locally commit. Likewise, if Q is in state ABORT, P can safely abort as well.

Now suppose that Q is still in state INIT. This situation can occur when the coordinator has sent VOTE-REQUEST to all participants, but this message has reached P (which subsequently responded with a VOTE-COMMIT message),

State of Q	Action by P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

Figure 8.31: Actions taken by a participant P when residing in state READY and having contacted another participant Q.

but has not reached Q. In other words, the coordinator had crashed while multicasting `VOTE-REQUEST`. In this case, it is safe to abort the transaction: both P and Q can make a transition to state ABORT.

The most difficult situation occurs when Q is also in state READY, waiting for a response from the coordinator. In particular, if it turns out that all participants are in state READY, no decision can be taken. The problem is that although all participants are willing to commit, they still need the coordinator's vote to reach the final decision. Consequently, the protocol blocks until the coordinator recovers.

The various options are summarized in Figure 8.31.

To ensure that a process can actually recover, it is necessary that it saves its state to persistent storage. For example, if a participant was in state INIT, it can safely decide to locally abort the transaction when it recovers, and then inform the coordinator. Likewise, when it had already taken a decision such as when it crashed while being in either state COMMIT or ABORT, it is in order to recover to that state again, and retransmit its decision to the coordinator.

Problems arise when a participant crashed while residing in state READY. In that case, when recovering, it cannot decide on its own what it should do next, that is, commit or abort the transaction. Consequently, it is forced to contact other participants to find what it should do, analogous to the situation when it times out while residing in state READY as described above.

The coordinator has only two critical states it needs to keep track of. When it starts the 2PC protocol, it should record that it is entering state WAIT so that it can possibly retransmit the `VOTE-REQUEST` message to all participants after recovering. Likewise, if it had come to a decision in the second phase, it is sufficient if that decision has been recorded so that it can be retransmitted when recovering.

It may thus be possible that a participant will need to block until the coordinator recovers. This situation occurs when all participants have received and processed the `VOTE-REQUEST` message from the coordinator, while in the meantime, the coordinator crashed. In that case, participants cannot cooperatively decide on the final action to take. For this reason, 2PC is also referred to as a **blocking commit protocol**.

There are several solutions to avoid blocking. One solution is to use a multicast primitive by which a receiver immediately multicasts a received message to all other processes [Babaoglu and Toueg, 1993]. It can be shown that this approach allows a participant to reach a final decision, even if the coordinator has not yet recovered. Another solution is to use three instead of two phases, as we discuss in Note 8.13.

Note 8.12 (Advanced: 2PC outlined in Python)

An outline of the actions that are executed by the coordinator is given in Figure 8.32. The coordinator starts by multicasting a VOTE-REQUEST to all participants in order to collect their votes. It subsequently records that it is entering the WAIT state, after which it waits for incoming votes from participants.

```

1 class Coordinator:
2
3     def run(self):
4         yetToReceive = list(participants)
5         self.log.info('WAIT')
6         self.chan.sendTo(participants, VOTE_REQUEST)
7         while len(yetToReceive) > 0:
8             msg = self.chan.recvFrom(participants, TIMEOUT)
9             if (not msg) or (msg[1] == VOTE_ABORT):
10                 self.log.info('ABORT')
11                 self.chan.sendTo(participants, GLOBAL_ABORT)
12                 return
13             else: # msg[1] == VOTE_COMMIT
14                 yetToReceive.remove(msg[0])
15         self.log.info('COMMIT')
16         self.chan.sendTo(participants, GLOBAL_COMMIT)

```

Figure 8.32: The steps taken by the coordinator in a 2PC protocol.

If not all votes have been collected but no more votes are received within a given time interval prescribed in advance, the coordinator assumes that one or more participants have failed. Consequently, it should abort the transaction and multicasts a GLOBAL-ABORT to the (remaining) participants. Likewise, if only a single participant decides to abort the transaction, the coordinator will have to call off the transaction. If all participants vote to commit, GLOBAL-COMMIT is first logged and subsequently sent to all processes. Otherwise, the coordinator multicasts a GLOBAL-ABORT (after recording it in the local log).

After receiving a vote request, the participant does its work. All bets are off if its work failed, but otherwise, it will vote for committing the transaction. It records its decision in a local log and informs the coordinator by sending a VOTE-COMMIT message. The participant must then wait for the global decision. Assuming this decision (which again should come from the coordinator) comes in on time, it is simply written to the local log, after which it can be carried out (the latter is not shown in the code).

```

1  class Participant:
2      def run(self):
3          self.log.info('INIT')
4          msg = self.chan.recvFrom(coordinator, TIMEOUT)
5          if (not msg): # Crashed coordinator - give up entirely
6              decision = LOCAL_ABORT
7          else: # Coordinator will have sent VOTE_REQUEST
8              decision = self.do_work()
9              if decision == LOCAL_ABORT:
10                 self.chan.sendTo(coordinator, VOTE_ABORT)
11             else: # Ready to commit, enter READY state
12                 self.log.info('READY')
13                 self.chan.sendTo(coordinator, VOTE_COMMIT)
14                 msg = self.chan.recvFrom(coordinator, TIMEOUT)
15                 if (not msg): # Crashed coordinator - check the others
16                     self.chan.sendTo(all_participants, NEED_DECISION)
17                     while True:
18                         msg = self.chan.recvFromAny()
19                         if msg[1] in [GLOBAL_COMMIT, GLOBAL_ABORT, LOCAL_ABORT]:
20                             decision = msg[1]
21                             break
22                     else: # Coordinator came to a decision
23                         decision = msg[1]
24                 if decision == GLOBAL_COMMIT:
25                     self.log.info('COMMIT')
26                 else: # decision in [GLOBAL_ABORT, LOCAL_ABORT]:
27                     self.log.info('ABORT')
28             while True: # Help any other participant when coordinator crashed
29                 msg = self.chan.recvFrom(all_participants)
30                 if msg[1] == NEED_DECISION:
31                     self.chan.sendTo([msg[0]], decision)

```

Figure 8.33: The steps taken by a participant process in 2PC.

Figure 8.33 shows the steps taken by a participant. First, the process waits for a vote request from the coordinator. If no message comes in, the transaction is simply aborted. Apparently, the coordinator had failed.

However, if the participant times out while waiting for the coordinator's decision to come in, it executes a termination protocol by first multicasting a DECISION-REQUEST message to the other processes, after which it subsequently blocks while waiting for a response. When a decisive response comes in (possibly from the coordinator, which is assumed to eventually recover), the participant writes the decision to its local log and handles accordingly. Any request from another participant for the final decision is left unanswered as long as that decision is not known.

We keep every participant up-and-running after it decided to either commit or abort. It can then assist other participants in need of a decision after detecting that the coordinator had crashed. To this end, a participant blocks on incoming messages and returns its own decision when asked for. Note that we are actually providing an implementation that supports partially synchronous behavior: we

assume that timeouts can be applied as a mechanism to detect failures, but take into account that we may have mistakenly concluded that a server crashed.

Note 8.13 (Advanced: Three-phase commit)

A problem with the two-phase commit protocol is that when the coordinator has crashed, participants may not be able to reach a final decision. Consequently, participants may need to remain blocked until the coordinator recovers. Skeen [1981] developed a variant of 2PC, called the **three-phase commit protocol (3PC)**, that avoids blocking processes in the presence of fail-stop crashes. Although 3PC is widely referred to in the literature, it is not applied often in practice as the conditions under which 2PC blocks rarely occur. We discuss the protocol, as it provides further insight into solving fault-tolerance problems in distributed systems.

Like 2PC, 3PC is also formulated in terms of a coordinator and a number of participants. Their respective finite state machines are shown in Figure 8.34. The essence of the protocol is that the states of the coordinator and each participant satisfy the following two conditions:

1. There is no single state from which it is possible to make a transition directly to either a COMMIT or an ABORT state.
2. There is no state in which it is not possible to make a final decision, and from which a transition to a COMMIT state can be made.

It can be shown that these two conditions are necessary and sufficient for a commit protocol to be nonblocking [Skeen and Stonebraker, 1983].

The coordinator in 3PC starts with sending a VOTE-REQUEST message to all participants, after which it waits for incoming responses. If any participant votes to abort the transaction, the final decision will be to abort as well, so the coordinator sends GLOBAL-ABORT. However, when the transaction can be committed, a PREPARE-COMMIT message is sent. Only after each participant has acknowledged it is now prepared to commit, will the coordinator send the final GLOBAL-COMMIT message by which the transaction is actually committed.

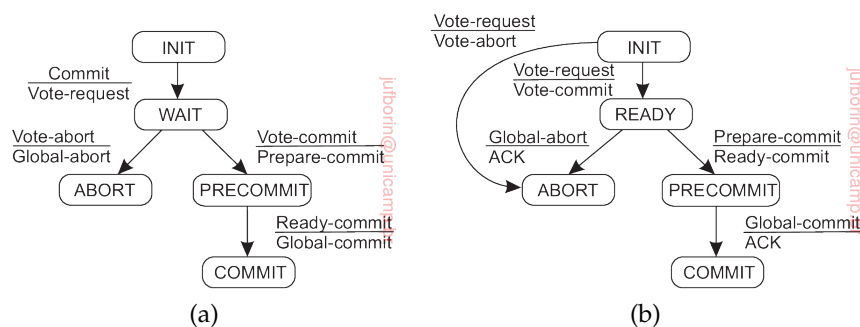


Figure 8.34: (a) The finite state machine for the coordinator in 3PC. (b) The finite state machine for a participant.

Again, there are only a few situations in which a process is blocked while

waiting for incoming messages. First, if a participant is waiting for a vote request from the coordinator while residing in state INIT, it will eventually make a transition to state ABORT, thereby assuming that the coordinator has crashed. This situation is identical to that in 2PC. Analogously, the coordinator may be in state WAIT, waiting for the votes from participants. On a timeout, the coordinator will conclude that a participant crashed, and will thus abort the transaction by multicasting a GLOBAL-ABORT message.

Now suppose the coordinator is blocked in state PRECOMMIT. On a timeout, it will conclude that one of the participants had crashed, but that participant is known to have voted for committing the transaction. Consequently, the coordinator can instruct the operational participants to commit by multicasting a GLOBAL-COMMIT message. In addition, it relies on a recovery protocol for the crashed participant to commit its part of the transaction when it comes up again.

A participant P may block in the READY state or in the PRECOMMIT state. On a timeout, P can conclude only that the coordinator has failed, so that it now needs to find out what to do next. As in 2PC, if P contacts any other participant that is in state COMMIT (or ABORT), P should move to that state as well. In addition, if all participants are in state PRECOMMIT, the transaction can be committed.

Again analogous to 2PC, if another participant Q is still in the INIT state, the transaction can safely be aborted. It is important to note that Q can be in state INIT only if no other participant is in state PRECOMMIT. A participant can reach PRECOMMIT only if the coordinator had reached state PRECOMMIT before crashing, and has thus received a vote to commit from each participant. In other words, no participant can reside in state INIT while another participant is in state PRECOMMIT.

If each of the participants that P can contact is in state READY (and they together form a majority), the transaction should be aborted. The point to note is that another participant may have crashed and will later recover. However, neither P, nor any other of the operational participants knows what the state of the crashed participant will be when it recovers. If the process recovers to state INIT, then deciding to abort the transaction is the only correct decision. At worst, the process may recover to state PRECOMMIT, but in that case, it cannot do any harm to still abort the transaction.

This situation is the major difference with 2PC, where a crashed participant could recover to a COMMIT state while all the others were still in state READY. In that case, the remaining operational processes could not reach a final decision and would have to wait until the crashed process recovered. With 3PC, if any operational process is in its READY state, no crashed process will recover to a state other than INIT, ABORT, or PRECOMMIT. For this reason, surviving processes can always come to a final decision.

Finally, if the processes that P can reach are in state PRECOMMIT (and they form a majority), then it is safe to commit the transaction. Again, it can be shown that in this case, all other processes will either be in state READY or at least, will recover to state READY, PRECOMMIT, or COMMIT when they had